

HDOC Day-10 Activity

- **Write a C Program to input time in seconds and convert it into days, hours, minutes and seconds format**

Logic

- 1 day = 86400 seconds
- 1 hour = 3600 seconds
- 1 minute = 60 seconds
- For total seconds S:
- Days = $S / 86400$
- Remaining seconds = $S \% 86400$
- Hours = Remaining seconds / 3600
- Remaining seconds = Remaining seconds % 3600
- Minutes = Remaining seconds / 60
- Seconds = Remaining seconds % 60

• Pseudocode:

START

Input total seconds S

Days = $S / 86400$

$S = S \% 86400$

Hours = $S / 3600$

$S = S \% 3600$

Minutes = $S / 60$

Seconds = S % 60

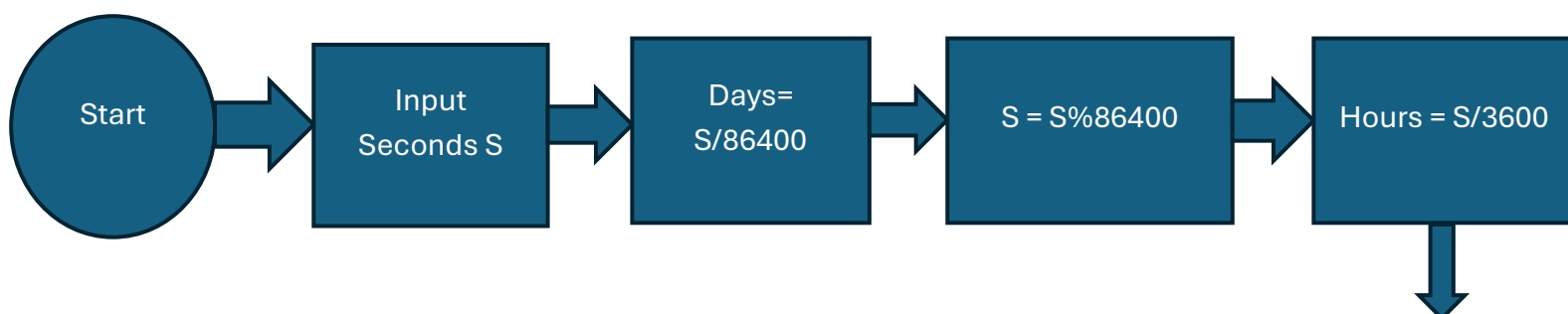
Display Days, Hours, Minutes and Seconds

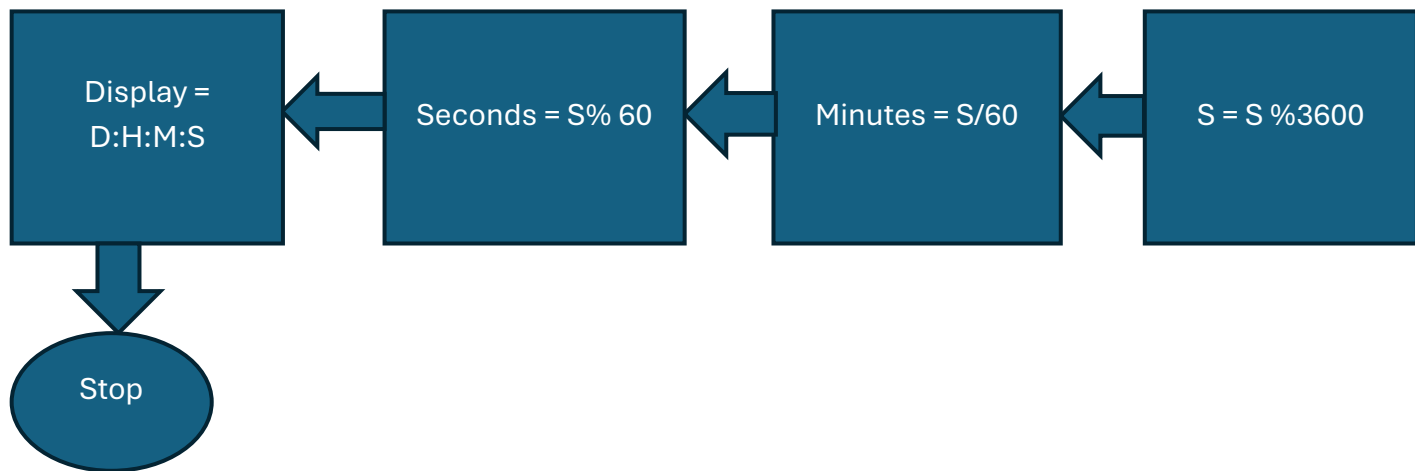
STOP

• Algorithm:

1. Start the program
2. Declare variables for total seconds, days, hours, minutes and seconds.
3. Input the total number of seconds.
4. Calculate days using seconds / 86400.
5. Find the remaining seconds using % 86400.
6. Calculate hours using remaining seconds / 3600.
7. Find the remaining seconds using % 3600.
8. Calculate minutes using remaining seconds / 60.
9. Calculate the remaining seconds using % 60.
10. Display days, hours, minutes and seconds.
11. Stop the program

Flowchart:





- **C Program:**

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    int totalSeconds, days, hours, minutes, seconds;
```

```
    printf("Enter time in seconds: ");
```

```
    scanf("%d", &totalSeconds);
```

```
    days = totalSeconds / 86400;
```

```
    totalSeconds = totalSeconds % 86400;
```

```
    hours = totalSeconds / 3600;
```

```
    totalSeconds = totalSeconds % 3600;
```

```
    minutes = totalSeconds / 60;
```

```
    seconds = totalSeconds % 60;
```

```
    printf("Days = %d\n", days);
```

```
    printf("Hours = %d\n", hours);
```

```
    printf("Minutes = %d\n", minutes);
```

```
    printf("Seconds = %d\n", seconds);
```

```
    return 0;
}
```

- **Test Case Table:**

Test Case	Input(Seconds)	Days	Hours	Minutes	
1	90061	1	1	1	
2	86400	1	0	0	
3	3661	0	1	1	
4	60	0	0	1	
5	0	0	0	0	

- **Compilation and Execution:** The code was successfully compiled and executed.

Test Case 1:

```
(kali㉿kali)-[~]
└─$ ./a.out
Enter time in seconds:90061
Days = 1
Hours = 1
Minutes = 1
Seconds = 1
```

Test Case 2:

```
(kali㉿kali)-[~]
└─$ ./a.out
Enter time in seconds:86400
Days = 1
Hours = 0
Minutes = 0
Seconds = 0
```

Test Case 3:

```
(kali㉿kali)-[~]  
$ ./a.out  
Enter time in seconds:3661  
Days = 0  
Hours = 1  
Minutes = 1  
Seconds = 1
```

Test Case 4:

```
(kali㉿kali)-[~]  
$ ./a.out  
Enter time in seconds:60  
Days = 0  
Hours = 0  
Minutes = 1  
Seconds = 0
```

Test Case 5:

```
(kali㉿kali)-[~]  
$ ./a.out  
Enter time in seconds:0  
Days = 0  
Hours = 0  
Minutes = 0  
Seconds = 0
```

Compilation and Execution Screenshots:

```
(kali㉿kali)-[~]  
$ vi scanner.c  
  
(kali㉿kali)-[~]  
$ cc scanner.c  
  
(kali㉿kali)-[~]  
$ ./a.out  
Enter time in seconds:90061  
Days = 1  
Hours = 1  
Minutes = 1  
Seconds = 1  
  
(kali㉿kali)-[~]  
$ ./a.out  
Enter time in seconds:86400  
Days = 1  
Hours = 0  
Minutes = 0  
Seconds = 0  
  
(kali㉿kali)-[~]  
$ ./a.out  
Enter time in seconds:3661  
Days = 0  
Hours = 1  
Minutes = 1  
Seconds = 1
```

Result:

The program successfully converts the given number of seconds into days, hours, minutes, and seconds.

- **Write a C Program to input a year and check whether it is a leap year or not.**

Leap Year Condition:

A year is a leap year if:

(year % 400 == 0)

OR

(year % 4 == 0 AND year % 100 != 0)

- **Pseudocode:**

START

Input year

IF year % 400 == 0

Display "Leap Year"

ELSE IF year % 4 == 0 AND year % 100 != 0

Display "Leap Year"

ELSE

Display "Not a Leap Year"

STOP

Algorithm:

Start the program

Declare an integer variable year.

Input the year.

Check whether the year is divisible by 400.

If yes, display Leap Year.

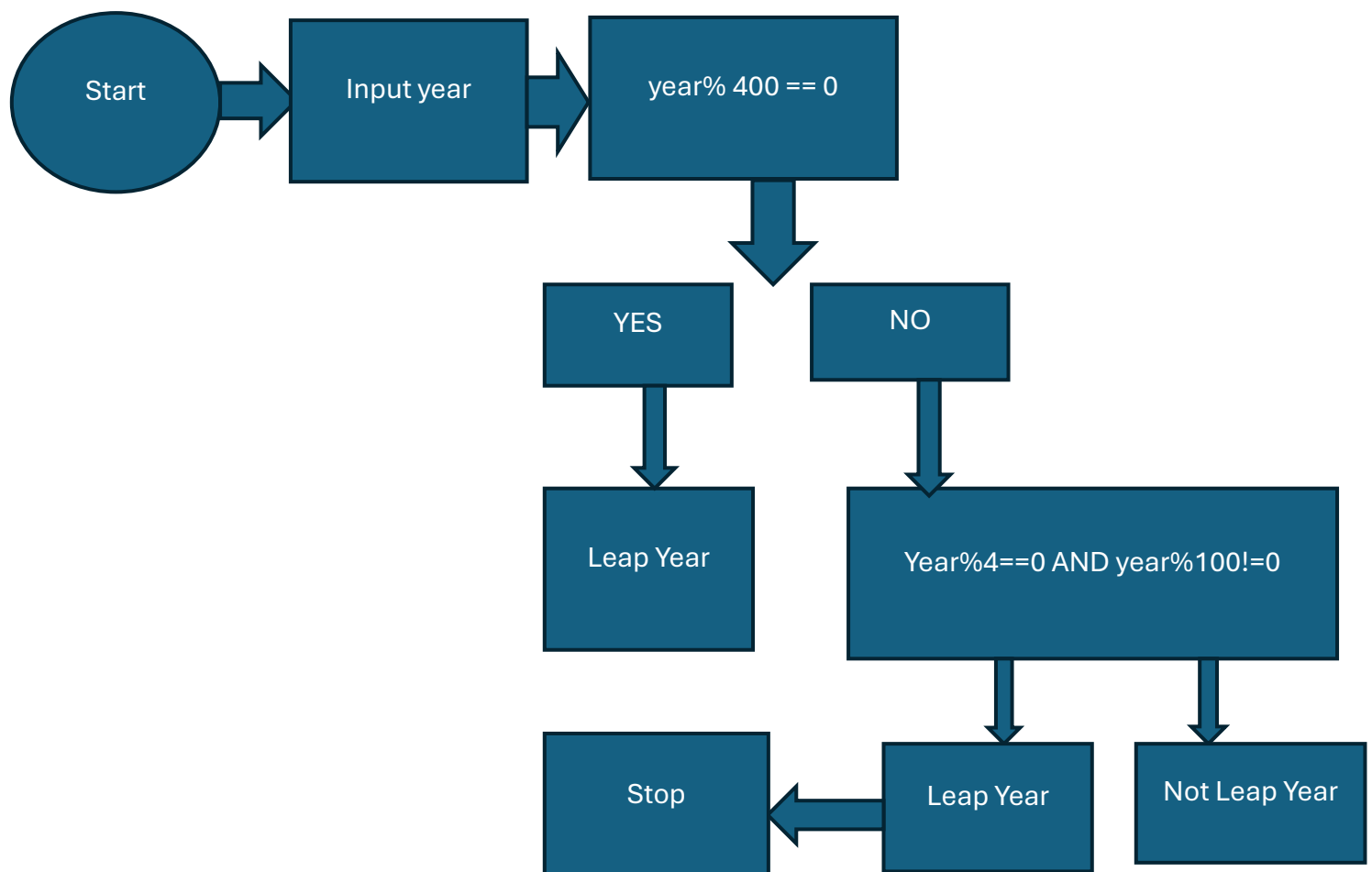
Otherwise, check whether the year is divisible by 4 and not divisible by 100.

If yes, display Leap Year.

Otherwise, display Not a Leap Year.

Stop

Flowchart:



• C Program:

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    int year;
```

```
    printf("Enter a year: ");
```

```
    scanf("%d", &year);
```

```
    if (year % 400 == 0)
```

```
    {
```

```
        printf("%d is a Leap Year.\n", year);
```

```
    }
```

```
    else if (year % 4 == 0 && year % 100 != 0)
```

```
    {
```

```
        printf("%d is a Leap Year.\n", year);
```

```
    }
```

```
    else
```

```
    {
```

```
        printf("%d is Not a Leap Year.\n", year);
```

```
    }
```

```
    return 0;
```

```
}
```

Test Case Table:

Test Case	Input Year	Expected Output
1	2024	Leap year
2	2023	Not a leap year
3	2000	Leap year
4	1900	Not a leap year
5	1600	Leap year

Compilation and Execution:

Test Case 1:

```
(kali㉿kali)-[~]  
$ ./a.out  
Enter a year:      2024  
2024 is a Leap year.
```

Test Case 2:

```
(kali㉿kali)-[~]  
$ ./a.out  
Enter a year:2023  
2023 is Not a Leap Year.
```

Test Case 3:

```
(kali㉿kali)-[~]  
$ ./a.out  
Enter a year:2000  
2000 is a leap Year.
```

Test Case 4:

```
(kali㉿kali)-[~]  
$ ./a.out  
Enter a year:1900  
1900 is Not a Leap Year.
```

Test Case 5:

```
(kali㉿kali)-[~]  
$ ./a.out  
Enter a year:1600  
1600 is a leap Year.
```

Compilation and Execution Screenshots:

```
(kali㉿kali)-[~]  
$ vi bottle.c  
  
(kali㉿kali)-[~]  
$ cc bottle.c  
  
(kali㉿kali)-[~]  
$ ./a.out  
Enter a year: 2024  
2024 is a Leap year.  
  
(kali㉿kali)-[~]  
$ ./a.out  
Enter a year:2023  
2023 is Not a Leap Year  
  
(kali㉿kali)-[~]  
$ ./a.out  
Enter a year:2000  
2000 is a leap Year.  
  
(kali㉿kali)-[~]  
$ ./a.out  
Enter a year:1900  
1900 is Not a Leap Year  
  
(kali㉿kali)-[~]  
$ ./a.out  
Enter a year:1600  
1600 is a leap Year.
```